Multiplayer Game with Client-Side Prediction and Server Reconciliation on an Emulated Network

Harish R
Roll No. 2024506092
Dept. of Information Technology
MIT Campus, Anna University
Chennai, India

Kalanithi Maran S
Roll No. 2024506108
Dept. of Information Technology
MIT Campus, Anna University
Chennai, India

Loga Janani J
Roll No. 2024506011
Dept. of Information Technology
MIT Campus, Anna University
Chennai, India

Poovizhi J
Roll No. 2024506019
Dept. of Information Technology
MIT Campus, Anna University
Chennai, India

***Abstract*—Real-time multiplayer games must hide network delay, jitter, loss and limited bandwidth from the player while one server stays authoritative. This paper presents NoBu Shooter, a browser-based arena shooter built to study these problems, and the algorithms inside it: client-side prediction with server reconciliation over a deterministic 60 Hz simulation, snapshot interpolation against an estimated server clock, delta-compressed snapshots against acknowledged bases, state synchronisation with extrapolation, server-relayed deterministic lockstep, and fragmentation for 1200-byte UDP datagrams. A standalone, protocol-agnostic emulator (random and Gilbert–Elliott burst loss, duplication, a bandwidth-limited FIFO queue, latency, jitter and reordering) makes every algorithm measurable under controlled conditions. Prediction cut input-to-screen latency from 370 ms to 18 ms at 100 ms one-way delay. Delta snapshots needed 22 % of the bandwidth of full snapshots (93 vs. 427 kbps) and almost halved the acknowledgement delay under a 400 kbps cap (306 vs. 592 ms). A redundant lockstep window reduced stalled frames from 25.5 % to 4.2 % at 5 % loss.**

***Index Terms*—client-side prediction, server reconciliation, snapshot interpolation, delta compression, lockstep, network emulation, UDP, WebSocket.**

I. INTRODUCTION

A player expects a key press to move the character on the next frame, yet in an online game the server alone decides what happened. Delay measurably hurts player performance [10]. Between the two lie latency, jitter, packet loss and a finite link rate. Games therefore combine several techniques: the client *predicts* its own movement and later *reconciles* it with the server's verdict [1], other players are shown by *interpolation* between snapshots [2], state is sent as small *deltas* [2], and some designs replace state by inputs in a *deterministic lockstep* [3], [4]. Each technique trades latency, smoothness, bandwidth and robustness against one another, and the trade is only visible when the network is bad in a controlled, repeatable way.

This project builds a complete game, NoBu Shooter, and a separate emulator, so that these trade-offs can be measured. Its contributions are: (1) an authoritative 60 Hz server and a client that share one deterministic movement function; (2) four synchronisation models, full, delta, state and lockstep, selectable per connection at run time; (3) a game-agnostic WebSocket/UDP impairment emulator with seeded, reproducible randomness; and (4) a measured comparison of all of them. The emphasis of this paper is on the algorithms and the technology that carries them; screenshots are deliberately left out.

II. SYSTEM AND TECHNOLOGY

A. Architecture

Fig. 1 shows the deployment. The server core is independent of the transport: a WebSocket [9] adapter (port 8080) and a UDP [8] adapter (port 8081) feed the same `GameServer`. Browsers cannot open UDP sockets, so they use WebSocket; headless *network players* use real UDP. The emulator sits between clients and server as a set of *routes* (`ws:9000`, `udp:9100`), each forwarding to one target and giving every client its own session with independent uplink and downlink. Because the test machine's links are loopback, the emulator is the only source of impairment. Its control API and dashboard listen on 127.0.0.1 only and accept browser pages only from an allow-listed origin.

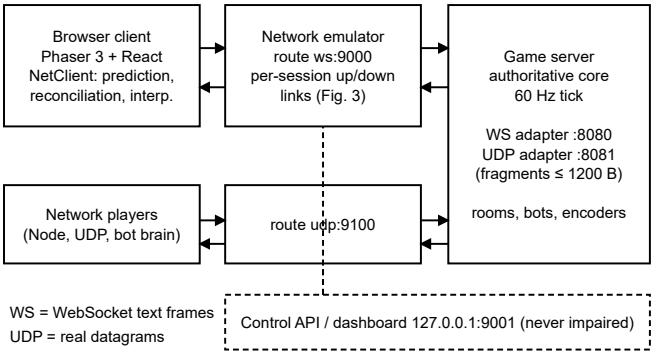


Fig. 1. System architecture. Browsers use WebSocket; network players use UDP. All data traffic can pass through the emulator; the control API is local only.

B. Technology stack

TABLE I — TECHNOLOGIES AND THEIR ROLE

Technology	Used for
TypeScript 5.5, Node.js ≥ 20	One language for client, server, emulator, tools
`ws` 8 (RFC 6455)	WebSocket server and emulator routes
Node `dgram`	UDP server adapter, emulator UDP routes, bots
Phaser 3.87 (WebGL)	Arena rendering, 60 fps
React 18, Zustand 4	HUD, Network Lab, Compare view
Vite 5	Dev server, production bundle
npm workspaces	`shared`, `server`, `emulator`, `client`
Vitest 1.6, playwright-core 1.48	284 tests; browser checks

The decisive design choice is the `shared` package. It holds the movement step, weapon and power-up rules, the protocol codec and the sync models. The server and the client import

the same functions, so a prediction on the client and the authoritative result on the server are bit-identical when they receive the same inputs. The step uses a fixed $dt = 1/60$ s, an integer key bitmask, `SQRT1_2` for diagonals and no trigonometry, random numbers or clock reads.

C. Protocol

Messages are UTF-8 JSON, one per WebSocket frame or UDP datagram (readable in the packet log; a binary encoding is future work). The client samples input at 60 Hz and sends batches at 30 Hz; each input has a sequence number s, a key mask, an aim angle and flags. The server sends snapshots at 30 Hz carrying `ack`, the last input it consumed. The handshake is built for datagrams: `hello` is repeated every 250 ms until `welcome` arrives, stale or duplicate inputs ($s \leq$ last consumed) are ignored, and a silent peer is dropped after 5 s.

III. ALGORITHMS

A. Authoritative tick

The server consumes exactly one input per player per tick in sequence order and skips gaps, then simulates movement, shots, hits and power-ups. If a backlog of more than 3 inputs builds up it consumes up to 2 per tick to catch up. Its output is the world state plus the player's own `ack`.

B. Client-side prediction and server reconciliation

Waiting for the server would delay every key press by a round trip. The client therefore applies each input to its own copy at once (prediction) and keeps unacknowledged inputs in a list. When a snapshot arrives (Fig. 2, Algorithm 1) it takes the authoritative position, discards inputs with $s \leq$ ack, replays the remaining ones with the shared `stepPlayer` and compares the result with its predicted position. With no loss the difference is exactly zero. Inputs lost on the uplink make the server skip steps, so the replay differs and the client must correct itself.

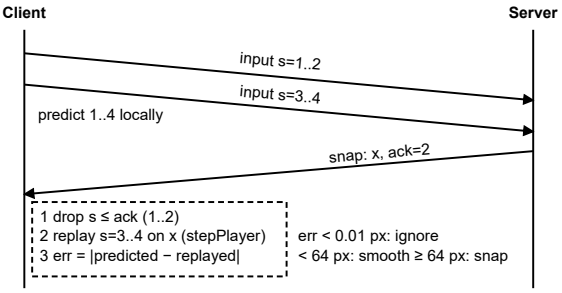


Fig. 2. Reconciliation: the client replays the inputs the server has not yet acknowledged on top of the authoritative position and compares the result with what it predicted.

Algorithm 1 `Reconciliation on snapshot S`

```
1  if S.me.life != life: hard reset; return
2  pending ← { i in pending : i.s > S.ack }
3  pos ← S.me.(x,y)
4  for i in pending: pos ← stepPlayer(pos, i.keys)
5  e ← | predicted − pos |
6  if e > 0.01 px:
7     if e < 64 px: offset ← offset + (predicted − pos)
8     else:         offset ← 0            (snap)
9  predicted ← pos
10 each tick: offset ← offset · 0.5^(dt/80 ms)
```

The visible error is smoothed rather than teleported: the correction becomes an offset that decays exponentially with a half-life T of 80 ms, $o(t+\Delta) = o(t)\cdot 2^{-\Delta/T}$. Errors below 0.01 px are ignored and errors of 64 px or more snap at once. Weapon state (ammo, reload, power-up timers) is reconciled the same way, because all timers advance per *consumed input* rather than per tick, so replay reproduces the server exactly. Optionally, the client sends the last up to 10 unacknowledged inputs in every message (*input redundancy*), which covers a lost message at a bandwidth cost.

C. Clock estimation and snapshot interpolation

Remote players cannot be predicted, so they are drawn from received snapshots. The client estimates the server clock offset from each snapshot, $\theta = st - arrival$, smoothed with an exponential average ($\alpha = 0.05$). It renders remote entities at $t_r = now + \theta - 100$ ms and linearly interpolates between the two snapshots that bracket t_r: $p = p_0 + (p_1 - p_0)(t_r - t_0)/(t_1 - t_0)$. The 100 ms buffer (three snapshot intervals) lets motion stay smooth across one lost snapshot and jitter. Respawns are never interpolated across the arena.

D. Delta snapshots

A full snapshot is 1.3–4.5 kB, yet most fields do not change between snapshots. Algorithm 2 sends only what changed since a snapshot the client has *acknowledged*. The server keeps a ring of the last 32 snapshots sent to each connection; the client reports the newest tick it decoded (`sa` on its inputs, or `snapAck`). A delta names its base, lists changed fields per entity, ids that disappeared (`gone`) and removed optional fields (`del`). If no acknowledged base is younger than 1 s the server falls back to a full snapshot. Because every delta states its base, loss, duplication and reordering cannot corrupt the client: a delta with an unknown base is dropped, never half-applied. Events are repeated in each delta until an acknowledged snapshot contained them. The cost is proportional to the amount of change instead of the size of the world.

Algorithm 2 `Delta encoding for one connection`

```
on send(S_t):
  B ← newest ring entry with tick ≤ ackedTick
  if B exists and age(B) ≤ 1 s:
      send snapDelta{ base:B.tick, tick:t,
         diff(B, S_t), gone, del, unconfirmed events }
  else: send full snap(S_t)            (fallback)
  ring.push(S_t)        (keep 32)
on ack(k): ackedTick ← max(ackedTick, k)
```

E. State synchronisation with extrapolation

State sync trades bytes for freshness. Complete records plus each entity's velocity (measured over the last tick; a jump above 30 px, such as a respawn, counts as 0) arrive at 10 or 30 Hz. The client draws remote entities at the estimated *current* server time, with no interpolation delay: $\hat{x}(t) = x_s + v(t - t_s)$, with the elapsed time clamped to $[-100, 250]$ ms, players pushed out of walls, and the jump at each new state blended away with a 100 ms half-life. Extrapolation is accurate on straight paths but overshoots at turns.

F. Deterministic lockstep

In lockstep the server sends *what players did*, not *where they are*. For each tick it relays the keys it consumed per player (30 Hz messages hold two ticks); every client applies them to its own world with the shared `stepPlayer`. A keyframe carries the whole world after events inputs cannot explain (join, leave, restart). The server stays the clock and never waits. A client plays ticks out two ticks behind its estimate of the server tick. If the next tick is missing the world freezes (a *stall*) and then catches up by at most 3 ticks

per frame. Lost ticks are recovered in one of two ways: *Rd off*, the client sends `lsNack` and retries after max(50 ms, 1.5×RTT); *Rd on*, every message repeats all records after the client's ack (at most 60). Every 30 ticks the server sends an FNV-1a hash of all positions; a mismatch counts as a *desync* and requests a keyframe. This mode is limited to the movement-only lab room, where the world is fully deterministic.

The hash check found a real portability bug: Chrome and Node differ in the last bit of `Math.cos`/`Math.sin` for 13,631 of 400,000 tested angles (3.4 %), which desynchronised the scripted drones. They now use `dsin`/`dcos` built only from +, −, ×, ÷ (exact under IEEE 754), after which 0 of 400,000 values differ and no desync occurred in later runs.

G. UDP fragmentation

A datagram is limited to 1200 B to stay below common path MTUs. Larger messages are split into ASCII-headed fragments `~id.i/n|` and reassembled per peer, accepting any order and discarding duplicates; an incomplete message is dropped after 250 ms or when more than 8 are pending. A message of k fragments survives with probability $(1 - p)^k$ (Fig. 6): one lost fragment loses the whole message. Full snapshots average 2.7 fragments in a four-player fight; delta and state messages almost always fit one.

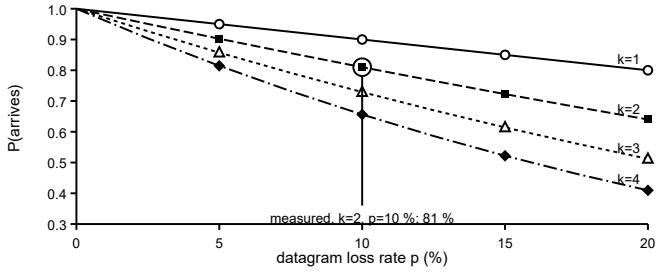


Fig. 6. Probability that a message split into k fragments is reassembled, (1 − p)^k.

H. Network emulator

Every packet (payload + 28 B of simulated UDP/IP header) passes the stages of Fig. 3. Random loss drops a packet if $u < p$. Burst loss is a two-state Gilbert–Elliott chain [5], [6] with leave-bad probability $q = 1/L$ (mean burst length L) and enter-bad probability $r = pq/(1-p)$; its stationary loss is $r/(r+q) = p$, so the configured percentage is the long-run loss while losses cluster. The bandwidth stage is a FIFO transmitter: a packet starts at max(*now*, *freeAt*), occupies the link for $8 \cdot size/rate$, and is tail-dropped if its queueing delay exceeds 400 ms. Delay is max(0, latency + J) with J normal (Box–Muller) or uniform. Ordering is kept by never releasing before the previous packet unless the reorder stage deliberately holds one back by 80 ms. All randomness comes from a seeded generator (mulberry32), so a run can be repeated.

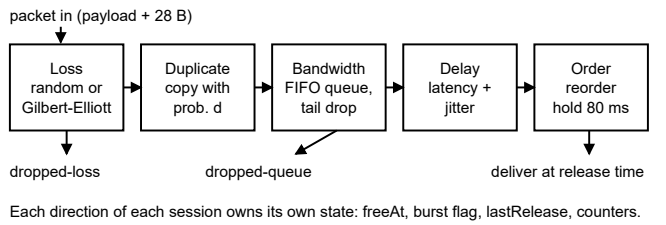


Fig. 3. Impairment pipeline applied to every packet, per session and direction.

IV. EXPERIMENTAL METHOD

All measurements ran on one Windows 11 laptop (Node 26.4, headless Chrome with the real GPU, 1920×1080) with every link on loopback. Conditions were set only through the emulator (Table II); latencies are one-way. The *Compare view* drives several panes from one shared 60 Hz input loop so that every pane sends identical inputs, differing in exactly one setting; the true state comes from an unimpaired spectator connection.

TABLE II — EMULATOR CONDITIONS USED

Name	Latency	Loss	Other
Baseline	0	0	–
Transatlantic	90 ± 8 ms	0.5 %	–
Nightmare	120 ± 50 ms	12 % burst	3 % dup, 5 % reorder, 400 kbps

Metrics: *Input→Screen* (key sample to the frame showing it); *ack delay* (input creation to the snapshot acknowledging it); *corrections/s*; *mover lag* (median delay at which the true path of a scripted drone passes through its drawn position, with its robust spread) and *frozen frames*; *off-path %* (frames where a drone is > 3 px from its true path at every delay, i.e. overshoot); downlink *kbps* on the wire; *stalled %* and *desyncs* for lockstep. Figures are single timed runs of 8–15 s per condition (ranges where repeated); no confidence intervals were computed.

V. RESULTS AND ANALYSIS

A. Prediction and reconciliation

TABLE III — PREDICTION AND RECONCILIATION

Condition	Setting	Result
Baseline	prediction on	RTT 17 ms, 0 corr./s, I→S 18 ms
100 ms one-way	prediction off	RTT 213 ms, I→S 370 ms
100 ms one-way	prediction on	I→S 18 ms, 0 corr./s
50 ms, 10 % loss	redundancy off	2–3 corr./s, ≈ 7 px
50 ms, 10 % loss	redundancy on	0 corr./s
50 ms, 10 % loss	reconciliation off	position drifts (≈ 40 px in test harness)

With prediction, Input→Screen no longer depends on the network: it fell from 370 ms to 18 ms (one frame), a 20-fold reduction. Latency alone caused *no* corrections; they appear only when inputs are lost, because the server then skips steps the client already performed. Redundancy removes the corrections at the price of resending up to 10 inputs per message. Without reconciliation the error never heals, which shows why prediction needs the replay step: prediction supplies responsiveness, reconciliation supplies correctness.

B. Interpolation

At 50 ± 30 ms of jitter, drones drawn directly from snapshots showed a lag of 75 ± 26 ms and were frozen in 60–70 % of frames; with interpolation the lag was 160 ± 3 ms with 2 % frozen frames. The 160 ms is the 50 ms network delay plus the fixed 100 ms buffer: interpolation exchanges about 85 ms of extra delay for an almost nine-fold smaller spread (26 to 3 ms) and 30-fold fewer frozen frames. Frozen frames also depend on the display: at 60 Hz showing 30 Hz snapshots, every second frame repeats, so about 50 % is the floor without interpolation.

C. Bandwidth and synchronisation models

TABLE IV — SYNC MODELS UNDER NIGHTMARE (≈ 15 S)

Model	↓ kbps	Ack ms	Off-path	Utilisation ρ
Full	427	592	6 %	> 1 (capped)
Delta	93	306	5 %	≈ 0.23
State 10 Hz	205	372	32 %	≈ 0.5

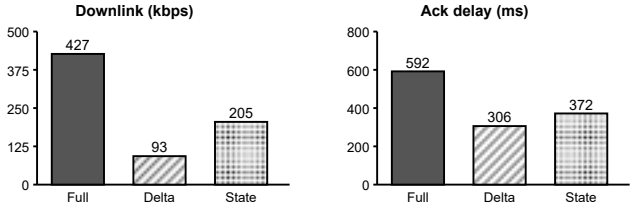


Fig. 4. Sync models under the Nightmare preset (400 kbps cap). Full = solid, Delta = hatched, State 10 Hz = dotted.

Under the 400 kbps cap, full snapshots (about 450–510 kbps offered in a four-player match) overload the link, ρ = offered/capacity > 1. The FIFO queue grows until tail-drop, so bytes turn into delay: this is bufferbloat [7]. Delta needs 22 % of the bytes, ρ stays near 0.23, the queue stays empty and the ack delay (306 ms) approaches its floor of roughly twice the 120 ms latency plus batching. A clean 300 kbps link without any loss isolates this effect: ack delay 562 vs. 145 ms and 297 vs. 77 kbps. The 417 ms difference is almost exactly the 400 ms queue limit, so the whole penalty is queueing. The lab scene shows the same ratio before any network effect: 1.85 kB per full snapshot against 0.36 kB per delta (19 %).

State sync at 10 Hz uses less than full (205 kbps) but extrapolation overshoots at direction changes: 32 % off-path against 5–6 % for interpolated models. Raising it to 30 Hz cost 3× the bytes (112 vs. 331 kbps) for 2.4× fewer off-path frames (31 vs. 13 %). Fragmentation compounds the advantage of delta over UDP: at 10 % loss a 2.7-fragment full snapshot is lost with probability $1 - 0.9^{2.7} \approx 25$ %, a one-fragment delta with 10 % (Fig. 6).

D. Snapshot versus lockstep

TABLE V — LOCKSTEP AT 50 MS ONE-WAY, 5 % LOSS

Model	Stalled	Longest	I→S ms	↓ kbps	Desync
Snapshot (full)	–	–	18	428	–
Lockstep, Rd off	25.5 %	295 ms	210	40	0
Lockstep, Rd on	4.2 %	37 ms	204	111	0

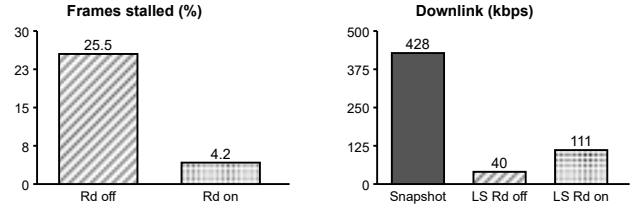


Fig. 5. Snapshot vs. lockstep (LS) at 50 ms one-way and 5 % loss. Rd = redundant window.

Lockstep sends about one tenth of the bytes of snapshots (40 vs. 428 kbps) but cannot hide loss: the world waits for every tick. With 30 messages/s and 5 % loss, about 1.5 records are lost per second; a measured 25.5 % stalled time implies a mean recovery of 0.255/1.5 ≈ 170 ms, consistent with a NACK round trip (retry after ≥ 1.5×RTT) plus resend. With Rd on the next message already carries the missing tick, so a stall needs several consecutive losses: 6× fewer stalled frames and an 8× shorter worst stall, for 2.8× the bytes. The residual 4.2 % exceeds the 0.25 % expected from two consecutive losses; timer granularity on Windows (≈ 15.6 ms) and clock-estimate error are the likely causes, though this was not isolated. Input→Screen is about 205 ms because lockstep here has no prediction: a key press is visible only after the round trip plus the two-tick playout buffer. Under 10 % *burst* loss even Rd on stalled about 17 % of frames, since four lost messages in a row outlast any buffer. Zero desyncs were recorded in every run, including a 2000-tick test in which the replay equalled the server bit for bit.

E. Summary and limitations

TABLE VI — WHAT EACH TECHNIQUE BUYS

Technique	Hides	Cost
Prediction + reconciliation	own latency (370→18 ms)	CPU, corrections
Input redundancy	uplink loss	extra bytes
Interpolation	jitter, snapshot loss	+100 ms remote lag
Delta snapshots	bandwidth (−78 %)	acks, base logic
State + extrapolation	remote lag	overshoot, 3× bytes at 30 Hz
Lockstep	bandwidth (−90 %)	stalls on loss

The results come from one machine and short single runs, so absolute numbers (for example the Windows timer granularity) will differ elsewhere; the ratios are the finding. The emulator drops packets without retransmission, so it models UDP; browsers actually use TCP, whose retransmission and head-of-line blocking are absent from these measurements and negligible only on a LAN. Lockstep covers just the movement-only lab room.

VI. CONCLUSION

Client-side prediction with server reconciliation, built on one shared deterministic step, removes network latency from the player's own control (370 to 18 ms) and is robust to lost inputs when combined with redundancy. Interpolation buys smooth remote motion for a fixed 100 ms. When bandwidth is scarce, acknowledged delta snapshots matter more than anything else: they cut bytes by 78 % and, through shorter queues, nearly halve the acknowledgement delay. Lockstep is the most economical but needs redundancy or retransmission to survive loss. The emulator, with seeded Gilbert–Elliott bursts and a queueing model, made these effects reproducible and explainable. Future work includes a binary protocol, a browser datagram transport (WebRTC or WebTransport) and measurements across real networks.

REFERENCES

[1] Y. W. Bernier, "Latency compensating methods in client/server in-game protocol design and optimization," *Game Developers Conf.*, 2001.

[2] G. Fiedler, "Snapshot interpolation" and "Snapshot compression," *Gaffer On Games*, 2014–2015. [Online]. Available: gafferongames.com

[3] G. Fiedler, "Deterministic lockstep," *Gaffer On Games*, 2014.

[4] P. Bettner and M. Terrano, "1500 archers on a 28.8: Network programming in Age of Empires and beyond," *Game Developers Conf.*, 2001.

[5] E. N. Gilbert, "Capacity of a burst-noise channel," *Bell Syst. Tech. J.*, vol. 39, no. 5, pp. 1253–1265, 1960.

[6] E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," *Bell Syst. Tech. J.*, vol. 42, no. 5, pp. 1977–1997, 1963.

[7] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *Commun. ACM*, vol. 55, no. 1, pp. 57–65, 2012.

[8] J. Postel, "User datagram protocol," IETF RFC 768, 1980.

[9] I. Fette and A. Melnikov, “The WebSocket protocol,” IETF RFC 6455, 2011.

[10] M. Claypool and K. Claypool, “Latency and player actions in online games,” *Commun. ACM*, vol. 49, no. 11, pp. 40–45, 2006.